Mechanistic Governance and Agent Configuration: A Compliance Architecture for Stela R-AGENT-XX

I. Executive Summary: The Mechanistic Doctrine in Practice

The governance of autonomous and semi-autonomous software repositories represents a fundamental paradigm shift from human-centric process management to machine-enforceable constraints. In traditional software engineering, governance is often a passive layer—a set of guidelines or "best practices" that rely on human compliance and institutional memory. The Stela system, as evidenced by its repository structure and constitutional documents, rejects this passive model in favor of a "Mechanistic Doctrine." This doctrine asserts that the system must prevent non-compliant actions mechanistically rather than requesting compliance through policy documents alone.[1] The operational reality of the Stela environment is one where policy is code, and code is policy, enforced by a rigid set of binaries, linters, and "Repo Gates" that act as the immune system of the platform.

The objective of this comprehensive report is to research, propose, plan, and discuss the update of docs/library/agents/R-AGENT-XX to conform to these rigorous operational specifications. This is not a trivial task of text editing; it is a structural engineering challenge that requires a nuanced understanding of the "Tiered Jurisdiction" that strictly separates the Platform (Core) from the Projects (Federation). The agent—whether it be an Architect, Integrator, or Contractor—must function within this jurisdictional framework without triggering the immediate enforcement actions, such as the rejection of Pull Requests or the termination of processes, that characterize the system's "Integrator plus Automation" layer.[1]

This analysis synthesizes the "Policy of Truth" (PoT), the repository's single source of truth, to derive the constraints for agent behavior. It examines the "Context Hazard" principle, which mandates the exclusion of library artifacts from the global context to prevent hallucination and logic drift. Furthermore, it analyzes the "Skills System," a subsystem designed to capture provenance and ensure that production payloads are generated through verifiable, automated harvesting rather than manual creation. The proposed plan for updating R-AGENT-XX integrates these elements, ensuring that the new agent definition adheres to "Linguistic Precision," "Context Hygiene," and "Absolute Literalism".[1] By adhering to the "Reuse-first" axiom, the proposal leverages existing operational binaries—specifically ops/bin/dump for state observation and ops/bin/project for secluded workflows—to create an agent definition that is not merely a text prompt, but a configured component of the Stela

runtime kernel.

II. The Stela Constitutional Framework

2.1 The Policy of Truth (PoT) as Absolute Law

The foundation of the Stela repository is the PoT.md, or Policy of Truth. This document is not a guideline; it is the constitution. Its preamble establishes the "Mechanistic Doctrine," stating explicitly that the system "prevents action that violates policy" and "does not request compliance".[1] This distinction is critical for the update of R-AGENT-XX. Any proposed update must be verifiable by the system's automated gates; if the update relies on human goodwill, memory, or "vibes," it is by definition non-compliant. The PoT operates as the supreme law of the repository, superseding all other documentation, including legacy files like SECURITY.md or CONTRIBUTING.md, which have been consolidated into the PoT to resolve jurisdictional ambiguity.[1]

The PoT establishes a hierarchy of authority known as "Jurisdiction." This concept is central to understanding where R-AGENT-XX fits within the system. The "Platform" scope, comprising ops/, docs/, tools/, and .github/, operates under absolute PoT enforcement. Here, no contractions are permitted, strict formatting is required, and the "Integrator" (the automated enforcement layer) has the mandate to stop work. The "Federation" scope, comprising projects/, operates under local law defined by a project-specific STELA.md, but it must strictly adhere to the platform's structural governance regarding file placement and interaction protocols.[1]

The implications of this for R-AGENT-XX are profound. As an artifact residing in docs/library/agents/, the agent itself is a Platform asset. Therefore, its internal definition—the prompt text that defines its behavior—must adhere to the strictest interpretation of Platform law. It cannot contain contractions. It cannot encourage "drift" or invention. It must be a model of the "Linguistic Precision" mandated by the constitution.

2.1.1 The Axioms of Stela

The governance model relies on a set of axioms that dictate decision-making logic, which must be encoded into the agent's operational instructions:

- **Precedence:** PoT is the final authority. In any conflict, the Operator (and by extension, the Agent) must "stop and ask" rather than inferring intent. This is a safety mechanism to prevent autonomous agents from hallucinating permissions or overriding safety checks.
- **SSOT (Single Source of Truth):** There is exactly one canonical file per domain. For governance, it is PoT.md; for history, it is SoP.md; for active work, it is TASK.md. Duplication of truth is considered "Drift" and is a system failure state. R-AGENT-XX must be instructed to never duplicate instructions found in these files but to reference them via pointers.
- **Reuse-First:** Before creating new logic, agents must search the ops/ directory for

existing templates. This prevents the proliferation of redundant scripts and ensures that R-AGENT-XX utilizes established binaries like ops/bin/dump rather than inventing new state-capture mechanisms.[1]

- **Context Hazard:** Perhaps the most technically specific axiom, this rule states that any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure. This forces a "Just-In-Time" (JIT) loading architecture, which the agent must be designed to support.

2.2 The Filing Doctrine and Directory Authority

To propose a valid update for R-AGENT-XX, one must understand the "Filing Doctrine" which assigns semantic meaning to the repository structure. This doctrine is physically enforced by the file system and repo-gates, creating a "physics" of the repository that the agent must navigate.

Directory	Semantics	Function	Governance Status
ops/	**Run**	Binaries, manifests, automation scripts.	Canonical (Platform). Strictly controlled.
docs/	**Explain**	Manuals, rationale, library definitions.	Canonical (Platform). The home of R-AGENT-XX.
projects/	**Work**	Payload code, active development.	Federated (Local Law). The target of the agent's work.
storage/	**Trash**	Local artifacts, dumps, handoffs.	Non-Canonical (Ignored). Ephemeral work surface.

An update to R-AGENT-XX falls under docs/, categorizing it as an "Explanation" surface. However, because it defines the behavior of an active agent, it interacts heavily with ops/. The agent defined in R-AGENT-XX must be instructed to utilize the binaries in ops/ to interact with the system. It cannot "act" directly; it must act through the "Run" layer. For example, it should not run raw git commands if a wrapper like ops/bin/open exists to handle the freshness gates

and state capture.[1]

2.3 The Logic of Linguistic Precision

The "Behavioral Logic Standard" in PoT Section 4.2 imposes a strict linguistic constraint: "No contractions." This rule applies across all scopes for the Core Platform and is enforced by tools/lint/style.sh.[1] The rationale is "Linguistic Precision"—avoiding ambiguity and enforcing a formal, objective tone. In the context of AI agent prompts, this has a functional purpose beyond style. Large Language Models (LLMs) operate on token probabilities. Contractions (e.g., "don't") introduce ambiguity regarding negation that can be subtle but impactful in complex instruction sets. By enforcing "do not," the system reduces the probability of the agent misinterpreting a negative constraint.

For R-AGENT-XX, this means the prompt instructions within the file must be written in expanded English. This applies to every section of the agent definition: its description, its role, its constraints, and its workflow instructions. An agent definition containing contractions is fundamentally non-compliant and will be rejected by the "Truth lint" gate, halting the update process before it can even be merged.[1]

III. The Architecture of Seclusion and Project Development

3.1 Defining Seclusion in the Stela Context

"Seclusion" in the Stela repository is not merely a privacy concept; it is an architectural enforcement of context isolation. It is the mechanism by which the system ensures that "Federated" project work does not contaminate the "Platform" core, and conversely, that the Platform does not impose unnecessary context overhead on specific projects. This is achieved through the rigorous application of the Filing Doctrine and the ops/bin/project binary.

Seclusion ensures that when an agent works on a specific project (e.g., projects/php-nuke-ce), it operates within a "Secluded Context." This context is artificially constructed to include only the necessary Platform laws (PoT, SoP) and the specific Project files. The rest of the repository—unrelated projects, deep library archives, and extraneous documentation—is mechanically excluded. This prevents "Context Drift," where an agent might conflate rules from one project with another, or hallucinate dependencies that exist elsewhere in the repo but not in the target project.[2]

3.2 The ops/bin/project Orchestrator

The technical implementation of seclusion is handled by the ops/bin/project binary. This tool acts as the gateway to the projects/ directory. According to the dump manifest [1] and the project.sh specifications, this binary performs several critical functions to enforce seclusion:

1. **Validation:** It verifies that the target project ID exists in the ops/lib/manifests/PROJECTS.md registry. This prevents agents from inventing projects or working on unregistered directories.
2. **Context Assembly:** It calls ops/bin/open to generate the freshness gate and ops/bin/dump to capture the platform state. Crucially, it then appends the project-specific STELA.md file.
3. **Handoff Generation:** It concatenates these elements into a single artifact in storage/handoff/.

For R-AGENT-XX, this means the agent definition must include instructions to *use* this binary. The agent cannot simply "cd into projects/foo". It must run ops/bin/project work foo to generate the secluded context file. This ensures that the agent enters the project scope through the authorized "airlock," respecting the jurisdictional boundaries.[1]

3.3 The Tiered Jurisdiction: Core vs. Federation

The update to R-AGENT-XX must explicitly address the "Tiered Jurisdiction" defined in PoT Section 3. The agent must be "jurisdiction-aware."

- **Platform (Core):** When the agent is performing maintenance on ops/ or docs/, it is under Platform Law. Strict PoT enforcement applies. No contractions. No deviations.
- **Federation (Projects):** When the agent moves into projects/, it enters a different jurisdiction. While the structural laws of the Platform still apply (e.g., no direct pushes to main), the *internal* coding standards are governed by the project's local STELA.md.

The agent definition must include a "Context Switch" logic. It must be instructed to read the local STELA.md immediately upon entering a project scope. This file serves as the "Local Constitution" for the project, potentially overriding Platform style guides (e.g., allowing contractions in a user-facing README if the project permits it).[1] Failure to encode this flexibility into R-AGENT-XX would result in an agent that rigidly enforces Platform rules on Federated projects, stifling development and violating the "Federation" principle.

3.4 Security Implications of Seclusion

Seclusion also serves a vital security function. By isolating project contexts, the system mitigates the risk of "Prompt Injection" or "Tool Misuse" spreading across the repository.[4] If a project contains malicious code or adversarial prompts, the seclusion boundaries prevent the agent from carrying that contamination back into the Platform core or into other projects. R-AGENT-XX must be designed to respect these boundaries as a security imperative, effectively operating as a "Sandboxed" entity when within the Federation scope.[5]

IV. Context Hygiene and Hazard Engineering

4.1 The Theory of Context Hazard

One of the most distinct and critical features of Stela governance is the concept of "Context Hazard." Section 1.2 of the PoT defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest".[1] This is not a warning; it is an axiom. Including these files is a "system failure."

The theoretical basis for this rule lies in the limitation of Large Language Model context windows and the danger of "Context Rot." If the global manifest (ops/lib/manifests/CONTEXT.md) were to include every agent definition and skill file, the LLM's context would be flooded with conflicting instructions. The specific rules for a "Code Reviewer" agent might clash with the rules for a "Planner" agent, leading to schizophrenic behavior where the agent attempts to satisfy contradictory directives simultaneously.

4.2 Just-in-Time (JIT) Loading Strategy

To manage this hazard, Stela employs a Just-in-Time (JIT) loading strategy. The "Negative Constraints" in docs/CONTEXT.md explicitly forbid the presence of library directories in the manifest.[1] Instead, agents and skills are loaded *only* when a Dispatch Packet (DP) explicitly requests them.

For the proposed update to R-AGENT-XX, this implies that the agent definition itself must not contain self-referential instructions that assume it is *always* present in the context. The agent must be designed to be "bootstrapped" into existence for a specific session and then discarded. It is a transient entity. The agent must be robust enough to operate with only the "Canonical Context" (PoT, SoP, TASK) and its own definition file, without relying on the presence of other sibling agents unless they are also explicitly loaded in the DP.

4.3 Automated Enforcement of Hygiene

The integrity of this JIT system is maintained by tools/lint/context.sh and ops/bin/llms.

- **tools/lint/context.sh:** This script scans the global manifest and fails if it detects any paths pointing to the forbidden library directories. If R-AGENT-XX were mistakenly added to ops/lib/manifests/CONTEXT.md by an Operator, this linter would trigger a failure state, blocking the PR.[1]
- **ops/bin/llms:** This binary, the "Context Bundler," enforces "Hazard Exclusion" at the generation level. When it builds the llms-full.txt or llms-small.txt bundles, it actively filters out the forbidden paths, ensuring that even if the manifest is corrupted, the generated bundles remain clean.[1]

R-AGENT-XX must be aware of this enforcement. Its definition should include instructions to *verify* context hygiene. Before beginning work, the agent should check that it has not been inadvertently loaded with conflicting contexts, adhering to the "Verify before Act" pattern.

V. The Skills System Paradigm and Agent Analogies

5.1 The Provenance Mandate and the Skills System

The user request specifically mandates that the update to R-AGENT-XX use "similar processes to the established Skills system." To do this, we must first deeply analyze the Skills system as defined in docs/library/SKILLS.md and ops/lib/scripts/skill.sh.[1]

The Skills system operates under a "Provenance Mandate." The PoT states that "Contractor closeout skill harvesting uses ops/lib/scripts/skill.sh harvest for provenance" and explicitly "forbids manual creation of docs/library/skills markdown files".[1] This is a radical departure from standard documentation practices. A skill cannot simply be written; it must be *harvested* from a verifiable work session.

5.2 The Harvesting Workflow

The Skills lifecycle follows a strict sequence:

1. **Execution:** Work is performed under a DP.
2. **Harvesting:** The command ops/lib/scripts/skill.sh harvest is run. This script scrapes the TASK.md for a ### HARVEST section, ensuring that the skill is directly derived from the executed task.[1]
3. **Provenance Injection:** The script injects a provenance block containing the DP ID, Branch, HEAD Hash, and Timestamp. This creates an immutable link between the skill and the context in which it was created.
4. **Promotion:** The ops/lib/scripts/skill.sh promote command moves the draft to the library and registers it in docs/library/INDEX.md.

5.3 Applying the Skills Paradigm to Agents

To update R-AGENT-XX using "similar processes," we cannot simply edit the text file. We must treat the Agent definition as a "Meta-Skill." The Agent is, effectively, a bundle of skills and directives. Therefore, the update process for R-AGENT-XX should mimic the harvesting workflow to ensure provenance.

Proposed Agent-Update Workflow:

1. **Drafting via DP:** The new agent definition should not be written directly into docs/library/agents/. Instead, it should be drafted in a work/ branch as part of a DP (e.g., DP-OPS-UPDATE-AGENT).
2. **Verification:** The draft agent should be tested against the "Repo Gates" (Truth lint, Context lint) within that work branch.
3. **"Harvesting" (Manual Simulation):** Since skill.sh is hardcoded for the skills/ directory, we cannot use it directly for agents yet. However, we must simulate the *logic* of the script. The PR updating the agent must include a "Provenance Block" in the PR description or a RESULTS.md file, citing the DP ID and the specific insights that led to the update.

4. **Registration:** Just as skill.sh promote updates docs/library/INDEX.md, the update to R-AGENT-XX must include a simultaneous update to the Index to register the new version or capabilities of the agent.[1]

This approach satisfies the requirement to use "similar processes" by enforcing the *principles* of the Skills system (provenance, automation, registry integrity) even if the exact binary support for "Agent Harvesting" is not yet implemented.

VI. Operational Specification for R-AGENT-XX

Based on the research above, the following specification is proposed for the updated docs/library/agents/R-AGENT-XX. This specification transforms the agent from a passive text file into an active component of the mechanistic system.

6.1 Frontmatter and Metadata

The file must begin with YAML frontmatter that defines its operational parameters for the Integrator.

YAML

```yaml
---
name: R-AGENT-XX
description:
tools:
model: opus
---
```

- **Tools:** Explicitly listing ops/bin/dump and ops/bin/project authorizes the agent to use these binaries. This is critical for "Seclusion" (using project to enter project scope) and "Observation" (using dump to see file state).
- **Model:** Specifying opus aligns with the high-reasoning requirements of the PoT.[1]

6.2 Role Definition and Constitutional Allegiance

The body of the agent definition must open with a declaration of allegiance to the Constitution.

> "You are a **Contractor** operating under **Integrator** governance. Your actions are constrained by the **Policy of Truth (PoT.md)**. You adhere to the **Mechanistic Doctrine**: you do not request compliance; you verify it."

This framing sets the "collaborator stance" defined in ops/bin/open [1] and reinforces the Staffing Protocol.

6.3 Operational Constraints (The "Hard Rules")

The agent must be programmed with the "Hard Rules" derived from copilot-instructions.md and PoT.md:

1. **Linguistic Precision:** "Do not use contractions. Use 'do not' instead of 'don't'. Provide objective, quantitative feedback."
2. **No Direct Pushes:** "Do not attempt to push to main. All work occurs on work/* branches."
3. **No Invention:** "Do not invent repository structures or paths. Verify existence using ops/bin/dump."
4. **Context Hygiene:** "Do not hallucinate files. Check ops/lib/manifests/CONTEXT.md for the authorized context set."

6.4 Workflow Integration

The agent must be instructed on how to move through the Stela workflow using the filing doctrine:

- **Session Start:** "Confirm loaded context. You must explicitly acknowledge PoT.md, SoP.md, and TASK.md."
- **Execution:** "Use ops/bin/dump --scope=platform (or project) to observe the state. Do not rely on conversation history; rely on the dump."
- **Seclusion Transition:** "When working on a project, execute ops/bin/project work <id> to generate the secluded handoff artifact. Read the project's STELA.md for local laws."
- **Skill Integration:** "If you identify a reusable pattern, propose a Skill Harvest. Do not write the skill manual; use ops/lib/scripts/skill.sh harvest."
- **Closeout:** "Generate a receipt in storage/handoff/. Ensure SoP.md is updated if Canon was touched."

6.5 The Mandatory Closing Block

The definition must strictly enforce the "Mandatory Closing Block" requirement from TASK.md. The agent must be instructed that it cannot finish a task without outputting this block.

> "You must output the Mandatory Closing Block as defined in TASK.md. Failure to do so is an automatic disapproval. The block must include: Primary Commit Header, Pull Request Title, Pull Request Description, Final Squash Stub, Extended Technical Manifest, and Review Conversation Starter." [1]

VII. Implementation Strategy and Risk Analysis

7.1 The "Refresh + Draft DP" Workflow

To implement this update, we will use the B-TASK-08 "Refresh + Draft DP" task model.[1]

1. **Refresh:** The Operator runs ops/bin/open to generate the freshness gate.
2. **Draft:** A DP is created in TASK.md with the specific intent: "Update R-AGENT-XX to Phase 3 Conformity."
3. **Execution:** The work is performed on a work/agent-update branch.
4. **Verification:** The new agent definition is subjected to tools/lint/truth.sh to ensure no contractions were introduced.

7.2 Handling "Context Hazard" during Development

A unique risk exists during the development of an agent file. Since docs/library/agents is a forbidden directory for the global manifest, the agent *working* on the update cannot "see" the file it is updating via the global context. It must rely on ops/bin/dump --include-dir=docs/library/agents to "see" the target. The update plan must explicitly include this refiner step in the DP to allow the worker to access the hazard zone safely.

7.3 Risk Assessment: "Drift" vs. "Evolution"

The Mechanistic Doctrine distinguishes between "Drift" (unauthorized change) and "Evolution" (governed change).

- **Drift:** An agent inventing a new directory structure libs/ instead of ops/lib/. This is a failure.
- **Evolution:** An agent proposing a new ops/bin/ tool via a DP, updating TASK.md, and logging it in SoP.md.

R-AGENT-XX must be explicitly instructed to avoid Drift. The instruction "Do not invent repository structures or paths not defined in PoT.md" is the primary guardrail. However, the agent must also be empowered to facilitate Evolution by using the proper channels (DPs and SoP updates).

7.4 Security Implications

By restricting the agent to ops/bin/ tools, we limit the surface area for security failures. The agent cannot execute arbitrary shell commands; it must use the sanctioned binaries which have built-in safety checks (e.g., ops/bin/dump respects scope boundaries and ops/bin/project enforces seclusion). This enforces the "Principle of Least Privilege" mechanistically.

VIII. Conclusion and Next Actions

The research confirms that updating docs/library/agents/R-AGENT-XX is a high-stakes operation within the Stela governance framework. The agent is not just a text file; it is a

configured component of the "Integrator" machinery. It functions as a semi-autonomous entity that must navigate complex jurisdictional boundaries, respect rigorous hygiene standards, and actively participate in the system's provenance tracking.

The proposed update conforms to Stela operational specs by:

1. **Adhering to PoT:** Strictly following the Filing Doctrine and Staffing Protocol.
2. **Respecting Context Hygiene:** Explicitly acknowledging its status as a potential Context Hazard and operating only when invoked via JIT loading.
3. **Integrating with Skills:** Mandating the use of skill.sh for provenance, effectively treating agent behaviors as harvestable skills.
4. **Enforcing Seclusion:** Respecting the boundaries between Platform and Projects via ops/bin/project.

Next Action: Proceed to the "Drafting" phase. Create a Dispatch Packet (DP) in TASK.md to formally commission this update, ensuring SoP.md is included in the scope to log the governance change. Run ops/bin/open to generate the session context and begin the work on a work/ branch.

Appendix A: Forensic Verification of Repository State

The following table summarizes the forensic state of the repository as derived from the provided dump, confirming the active governance constraints that dictate this update.

Surface	State	Governance Implication
PoT.md	Active (SSOT)	Defines "Context Hazard" and "Mechanistic Doctrine." Final Authority.
SoP.md	Active (Ledger)	Must be updated in the same PR as any Canon change. 30-entry limit.
TASK.md	Active (Contract)	Defines the current DP. Gate surfaces change triggers policing.
ops/bin/open	Binary	Enforces freshness gates and context injection

		(Timekeeper).
ops/bin/dump	Binary	Enforces scope seclusion (Platform vs. Project) (Observer).
skill.sh	Script	Enforces provenance for skills; forbids manual creation.
context.sh	Linter	Enforces Context Hygiene (Hazard detection).
project	Binary	Enforces Seclusion and Federation boundaries.

Appendix B: The "Mandatory Closing Block" Requirement

Any work resulting from this report must conclude with the "Mandatory Closing Block" as defined in TASK.md. This ensures that the provenance of the update is cryptographically and semantically distinct.

1. **Primary Commit Header:** Plaintext summary.
2. **Pull Request Title:** distinct from header.
3. **Pull Request Description:** Markdown detailed summary.
4. **Final Squash Stub:** distinct from header.
5. **Extended Technical Manifest:** detailed list of changes.
6. **Review Conversation Starter:** specific questions for the reviewer.

Failure to provide this block constitutes an "Automatic Disapproval".[1]

Detailed Analysis of Stela Subsystems

1. The "Open" Subsystem (ops/bin/open)

The open binary is the "Timekeeper" of the repository. It captures the git status, HEAD hash, and active branch. For R-AGENT-XX, this means the agent must be aware that its "memory" of the repo is only as fresh as the open artifact. It cannot assume the repo state has not changed since the prompt was generated. This necessitates a "Verify before Act" pattern in

the agent's logic.

2. The "Dump" Subsystem (ops/bin/dump)

The dump binary is the "Observer." It provides the deterministic snapshot of files. R-AGENT-XX must rely on dump outputs to "see" the file system. It should not ask the user to "paste file contents" but rather request a dump operation. This ensures that the context loaded into the agent is identical to the disk state, preventing hallucination.

3. The "Project" Subsystem (ops/bin/project)

This binary orchestrates the workflow. It validates project IDs and assembles the context. R-AGENT-XX must utilize this tool when transitioning from the Platform scope to the Project scope. It enforces the "Seclusion" by ensuring that when working on a project, the agent only sees the relevant project files plus the platform core, rather than the entire repository. This protects the Platform from contamination by Project logic.

4. The "Prune" Subsystem (ops/bin/prune)

This binary manages the hygiene of the SoP.md and storage/ directories. R-AGENT-XX should be aware of this lifecycle. It should know that SoP entries are archived and that storage/handoff artifacts are ephemeral. This prevents the agent from relying on long-term storage of temporary artifacts.

Synthesis of Agent "R-AGENT-XX" Definition

The reconstructed definition for R-AGENT-XX, based on the strictures of PoT.md, represents a "compliance-first" entity. Unlike generic coding agents that prioritize speed or creativity, a Stela agent prioritizes **provenance** and **safety**.

- **Constraint 1:** It must never use the word "don't." It must say "do not."
- **Constraint 2:** It must never create a file in docs/library/skills/. It must run ops/lib/scripts/skill.sh.
- **Constraint 3:** It must never push to main. It must wait for the Operator.
- **Constraint 4:** It must never hallucinate a file. It must check ops/lib/manifests/CONTEXT.md.

This rigorous definition turns the agent into a reliable component of the mechanistic system, capable of operating autonomously within its secluded scope while remaining fully aligned with the constitutional requirements of the Policy of Truth.

References to Ground Truth

- **PoT.md:** Sections 1.1, 1.2, 2, 3, 4.1, 4.2.[1]
- **TASK.md:** Sections 3.4.5, 4, 5.[1]

- **SoP.md:** Archive policy and policing.[1]
- **.github/copilot-instructions.md:** Hard rules for guest contractors.[1]
- **docs/ops/specs/**: Technical specifications for binaries.[1]

This report confirms that the update to R-AGENT-XX is feasible, necessary, and rigorously defined by the existing repository canon. The path forward is clear, governed by the immutable logic of the Stela constitution.

Works cited

1. dump-full-main-9e291e78f.txt
2. Building an AI agent framework, running into context drift & bloated prompts. How do you handle this? : r/AI_Agents - Reddit, accessed February 6, 2026, https://www.reddit.com/r/AI_Agents/comments/1m1x1t6/building_an_ai_agent_framework_running_into/
3. 8 Tactics to Reduce Context Drift with Parallel AI Agents - Lumenalta, accessed February 6, 2026, https://lumenalta.com/insights/8-tactics-to-reduce-context-drift-with-parallel-ai-agents
4. LLM guardrails: Best practices for deploying LLM apps securely - Datadog, accessed February 6, 2026, https://www.datadoghq.com/blog/llm-guardrails-best-practices/
5. AI Coding Agent Security: Threat Models and Protection Strategies - Knostic, accessed February 6, 2026, https://www.knostic.ai/blog/ai-coding-agent-security